MIMIR

SMART CONTRACT AUDIT

ZOKYO.

October 15th, 2021 | v. 1.0

PASS

Zokyo's Security Team has concluded that this smart contract passes security qualifications to be listed on digital asset exchanges.

TECHNICAL SUMMARY

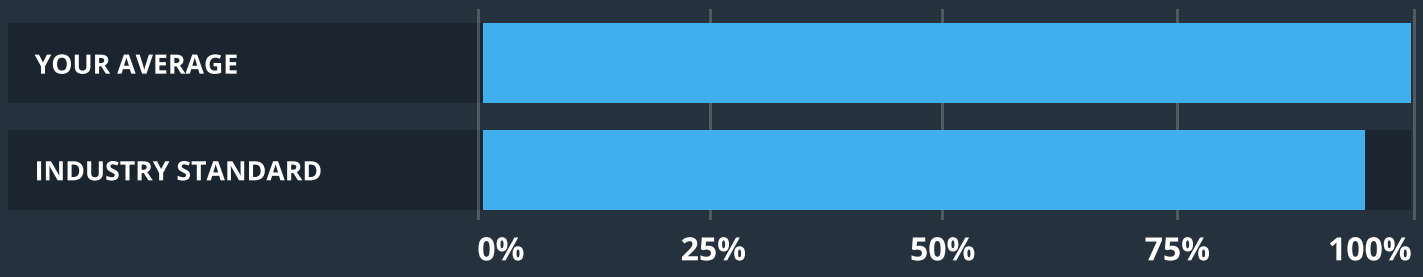
This document outlines the overall security of the MIMIR smart contracts, evaluated by Zokyo's Blockchain Security team.

The scope of this audit was to analyze and document the MIMIR smart contract codebase for quality, security, and correctness.

Contract Status

There were no critical issues found during the audit.

Testable Code



The testable code is 100%, which is above the industry standard of 95%.

It should be noted that this audit is not an endorsement of the reliability or effectiveness of the contract, rather limited to an assessment of the logic and implementation. In order to ensure a secure contract that's able to withstand the Ethereum network's fast-paced and rapidly changing environment, we at Zokyo recommend that the MIMIR team put in place a bug bounty program to encourage further and active analysis of the smart contract.

TABLE OF CONTENTS

Auditing Strategy and Techniques Applied 3
Summary 5
Structure and Organization of Document 6
Complete Analysis 7
Code Coverage and Test Results for all files 12
Tests written by Zokyo Secured team 12

AUDITING STRATEGY AND TECHNIQUES APPLIED

The Smart contract's source code was taken from the Gamestation repository.

Repository:
https://github.com/conexuz/Mimir-Token/commit/1a0a42188d3ee2abee595c1c260a5c9b76d83423

Last commit:
eaa79954ff77e16e93126497861921789c9a30b5

Contracts under the scope:
MimirToken.sol

Throughout the review process, care was taken to ensure that the token contract:

- Implements and adheres to existing Token standards appropriately and effectively;
- Documentation and code comments match logic and behavior;
- Distributes tokens in a manner that matches calculations;
- Follows best practices in efficient use of gas, without unnecessary waste;
- Uses methods safe from reentrance attacks;
- Is not affected by the latest vulnerabilities;
- Whether the code meets best practices in code readability, etc.

Zokyo's Security Team has followed best practices and industry-standard techniques to verify the implementation of MIMIR smart contracts. To do so, the code is reviewed line-by-line by our smart contract developers, documenting any issues as they are discovered. Part of this work includes writing a unit test suite using the Truffle testing framework. In summary, our strategies consist largely of manual collaboration between multiple team members at each stage of the review:

1	Due diligence in assessing the overall code quality of the codebase.
2	Cross-comparison with other, similar smart contracts by industry leaders.
3	Testing contract logic against common and uncommon attack vectors.
4	Thorough, manual review of the codebase, line-by-line.

SUMMARY

Zokyo team has conducted a smart contract audit for the given codebase. The scope of work and the contracts that are under the investigation are presented in Auditing Strategy and Techniques Applied section.

During the process, Zokyo security team has found informational issues and issues with low severity level. No critical issues were spotted. It is worth mentioning that most of them were fixed by the MIMIR team and only a couple of informational issues were left unresolved.

The contracts are in good condition. Based on the fixes provided by the MIMIR team and on the quality and security of the codebase provided, Zokyo team can give a score of 99 to the audited smart contracts.

We can state that the contracts bear no security or operational risk to the end-user or contract owner, so they are fully production-ready.

STRUCTURE AND ORGANIZATION OF DOCUMENT

For ease of navigation, sections are arranged from most critical to least critical. Issues are tagged “Resolved” or “Unresolved” depending on whether they have been fixed or addressed. Furthermore, the severity of each issue is written as assessed by the risk of exploitation or other unexpected or otherwise unsafe behavior:

Critical

The issue affects the ability of the contract to compile or operate in a significant way.

High

The issue affects the ability of the contract to compile or operate in a significant way.

Medium

The issue affects the ability of the contract to operate in a way that doesn’t significantly hinder its behavior.

Low

The issue has minimal impact on the contract’s ability to operate.

Informational

The issue has no impact on the contract’s ability to operate.

COMPLETE ANALYSIS

LOW | RESOLVED

Useless function getOwner() in the contract ERC20. To get the owner of the contract you can use the owner() function from the Ownable contract.

Recommendation:
Consider removing the getOwner() function to reduce the gas cost while the deployment.

LOW | RESOLVED

Solidity 0.8 includes a check of arithmetic operations by default, hence SafeMath is unnecessary.

Recommendation:
Consider removing the SafeMath library to optimize the contract size and gas usage.

INFORMATIONAL | UNRESOLVED

Multiple Contracts Per File. The Mimirtoken file contains few independent contracts, libraries, and interfaces: Context, Ownable, SafeMath, Address, IERC20, ERC20, and MIMIRToken. This does not follow the Solidity style guide and makes the code harder to read.

Recommendation:
Use the separate files for each contract or just use the imports of OpenZeppelin standard for the contract such as Context, Ownable, SafeMath, Address, IERC20, ERC20.

INFORMATIONAL | UNRESOLVED

The docstrings of the contracts and functions are partially following the Ethereum Natural Specification Format (NatSpec). Consider adding the relevant missing tags to all functions of the MIMIRToken contract.

INFORMATIONAL | UNRESOLVED

Unnecessary public visibility in some functions which are not used locally in the contracts.

Recommendation:
Change the public with external visibility for functions that are not used locally in the contracts to reduce gas costs.

INFORMATIONAL | RESOLVED

The _cap variable in the contract ERC20 is initialized while the contract is created and does not change anywhere in the contract code, so it is a constant value.

Recommendation:
Consider marking the _cap variable as CONSTANT.

Re-audit:
Not valid since the variable was deleted.

INFORMATIONAL | RESOLVED

According to the style guide function arguments' names should be in mixedCase.

Recommendation:
In the constructor of the ERC20 contract change the naming of the arguments from the CapWords to mixedCase as follow:

```
constructor(string memory name, string memory symbol) {}
```

INFORMATIONAL | RESOLVED

For more code clarity consider moving the extra added functionality from the ERC20 contract to the MIMIRToken contract such as cap() and public mint() function. And in this case move the require from _mint() function to public mint() function:

```
require(totalSupply().add(amount) <= cap(), "only 100 million MIMIR Exist");
```

INFORMATIONAL | RESOLVED

Follow the one style for error messages in the contracts.

Recommendation:
Consider changing the error message in ERC20 contract related to exceeding the cap as follow:

```
require(totalSupply().add(amount) <= cap(), "ERC20: only 100 million MIMIR Exist");
```

Re-audit:
Not valid since require was deleted

INFORMATIONAL | NOT VALID

Misleading comment in line 954, contract MMIRToken:

```
 * @notice Delegate votes from `msg.sender` to `delegatee`
 * @param delegator The address to get delegatee for
 /*
function delegates(address delegator)
    external
    view
    returns (address)
    return _blackAndWhitePrice;
{

    return _delegates[delegator];
}
```

Recommendation:
Consider changing the comment to describe what the function really do:

```
@notice Get the delegatee of the delegator
```

Re-audit:
Not valid since the function was deleted.

	MimirToken.sol
Re-entrancy	Pass
Access Management Hierarchy	Pass
Arithmetic Over/Under Flows	Pass
Unexpected Ether	Pass
Delegatecall	Pass
Default Public Visibility	Pass
Hidden Malicious Code	Pass
Entropy Illusion (Lack of Randomness)	Pass
External Contract Referencing	Pass
Short Address/ Parameter Attack	Pass
Unchecked CALL Return Values	Pass
Race Conditions / Front Running	Pass
General Denial Of Service (DOS)	Pass
Uninitialized Storage Pointers	Pass
Floating Points and Precision	Pass
Tx.Origin Authentication	Pass
Signatures Replay	Pass
Pool Asset Security (backdoors in the underlying ERC-20)	Pass

CODE COVERAGE AND TEST RESULTS FOR ALL FILES

Tests written by Zokyo Security team

As part of our work assisting MIMIR in verifying the correctness of their contract code, our team was responsible for writing integration tests using the Truffle testing framework.

Tests were based on the functionality of the code, as well as a review of the MIMIR contract requirements for details about issuance amounts and how the system handles these.

Code Coverage

The resulting code coverage (i.e., the ratio of tests-to-code) is as follows:

FILE	% STMTS	% BRANCH	% FUNCS	% LINES	UNCOVERED LINES
contracts\	100.00	100.00	100.00	100.00	
MimirToken.sol	100.00	100.00	100.00	100.00	
All files	**100.00**	**100.00**	**100.00**	**100.00**	

Test Results

```
Contract: MimirToken
  constructor
    ✓ should deploy with correct token name (114ms)
    ✓ should deploy with correct token symbol (113ms)
    ✓ should deploy with correct decimals (223ms)
    ✓ should deploy with correct total supply (131ms)
    ✓ should mint total supply to owner address correctly (1322ms)
    ✓ should deploy with correct owner address (86ms)
  functions:
    transfer
      ✓ should transfer correctly (311ms)
```

```
    ✓ should catch event
    ✓ cannot transfer from zero address (227ms)
    ✓ cannot transfer to zero address (304ms)
    ✓ should revert if amount exceeds balance (228ms)
  transferFrom
    ✓ should transferFrom correctly (643ms)
    ✓ should catch event
    ✓ cannot transfer from zero address (244ms)
    ✓ cannot transfer to zero address (256ms)
    ✓ should revert if amount exceeds balance (264ms)
  approve
    ✓ should approve tokens correctly (157ms)
    ✓ should catch event
    ✓ cannot approve to the zero address (242ms)
    ✓ cannot approve from the zero address (248ms)
  allowance
    ✓ should return allowance tokens correctly (125ms)
    ✓ should increase allowance correctly (303ms)
    ✓ should decrease allowance correctly (331ms)
    ✓ should revert if subtracted amount more than current allowance (193ms)
  transferOwnership
    ✓ should transfer ownership correctly (364ms)
    ✓ cannot transfer ownership to zero address (203ms)
    ✓ should revert if caller is not the owner (238ms)
  renounceOwnership
    ✓ should renounce ownership correctly (266ms)
  _burn
    ✓ should burn tokens correctly (328ms)
    ✓ should catch event
    ✓ cannot burn from the zero address (205ms)
    ✓ should revert if burn amount exceeds balance (231ms)
  _msgData
    ✓ should return msg.data correctly (99ms)

33 passing (14s)
```

We are grateful to have been given the opportunity to work with the MIMIR team.

The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

Zokyo's Security Team recommends that the MIMIR team put in place a bug bounty program to encourage further analysis of the smart contract by third parties.

ZOKYO.